



SUPPORT FORMATEUR · ÉDITION OCTOBRE 2026

Formation Prometheus & Grafana

Guide du formateur — Jour 3, version pratique : le déroulé pas à pas, le texte à dire, les manipulations, les corrections

Sommaire

Jour 3, version pratique — le guide pas à pas	2
La journée en un coup d'œil	2
La veille au soir (ou à 7h30) — 30 minutes	3
8h30 — Vérification	5
9h00 — Démarrer (15 min) · slides 1 à 5	6
9h15 — Les bonnes métriques et PromQL avec l'IA (25 min) · slides 6 à 13	7
9h40 — TP A : la boutique en quatre signaux dorés (50 min + 40 min) · slides 14 à 18	9
10h30 — Pause	11
11h00 — TP A, suite : les finitions (je les montre, 5 min)	11
11h40 — TP B : le serveur en méthode USE (30 min) · slide 19	12
12h10 — Auto-audit et maturité (20 min) · slides 20 et 21	13
12h30 — Déjeuner	13
13h30 — Alerter sans épuiser les équipes (10 min) · slides 22 à 27	13
13h40 — TP C : Prometheus, Alertmanager, Teams, Slack, e-mail (50 min) · slides 28 à 30	14
14h30 — Pause	16
15h00 — TP D : l'alerting de Grafana, Teams en vrai (30 min) · slides 31 à 33	16
15h30 — Bonnes pratiques et escape game (20 min) · slides 34 à 38	18
15h50 — Thanos en cinq minutes · slide 39	19
15h55 — Lundi matin · slide 40	20
Annexe A — La télécommande jour3.sh	21
Annexe B — Ce que contient la branche jour3	21
Annexe C — Dépannage express	22

Jour 3, version pratique — le guide pas à pas

Ce guide se suit à la lettre, de la veille au soir à 16h00. Pour chaque séquence : l'heure, les slides du deck **Jour3-Prometheus-Grafana.pptx** (40 slides), ce que je dis (en citations), ce que je fais (encadrés **Pas à pas**, sur mon Mac et dans les Codespaces), les corrections, les pièges et les anecdotes.

L'esprit de la journée. Le groupe vient de l'infrastructure. On n'écrit pas de code et on n'écrit pas de PromQL à la main : l'IA écrit la requête **et l'explique**, les stagiaires la comprennent et la vérifient. Tout le temps gagné va dans ce qu'ils feront lundi : des tableaux de bord, des alertes dans Teams et par e-mail, et le diagnostic d'une panne. La phrase à dire à 9h00 : **aujourd'hui, vous ne codez pas, vous pilotez.**

Le cadre. 9h00–16h00, déjeuner d'une heure, deux pauses d'une demi-heure : cinq heures de travail effectif, dont environ quatre les mains sur le clavier.

La journée en un coup d'œil

Heure	Durée	Séquence	Slides	Modalité
9h00	15 min	Démarrer : un Codespace sur la branche <code>jour3</code> , <code>./jour3.sh start</code>	1 à 5	Tous ensemble
9h15	25 min	Les bonnes métriques (Google SRE, RED, USE), PromQL avec l'IA, échauffement	6 à 13	Exposé, démo, individuel
9h40	50 min	TP A — La boutique en quatre signaux dorés (étapes 0 à 2)	14 à 18	Individuel
10h30	30 min	<i>Pause</i>		
11h00	40 min	TP A — métier, finitions, crash test	17	Individuel
11h40	30 min	TP B — Le serveur en méthode USE, import du 1860	19	Individuel
12h10	20 min	Auto-audit des dashboards, modèle de maturité	20, 21	Binômes
12h30	60 min	<i>Déjeuner</i>		
13h30	10 min	Alerter sans épuiser les équipes	22 à 27	Exposé
13h40	50 min	TP C — Prometheus, Alertmanager, Teams, Slack, e-mail	28 à 30	Individuel, démo Teams à la fin
14h30	30 min	<i>Pause</i>		

Heure	Durée	Séquence	Slides	Modalité
15h00	30 min	TP D — Alerting Grafana, Teams en vrai, comparaison	31 à 33	Individuel puis tous ensemble
15h30	20 min	Bonnes pratiques (2 slides), escape game « la boutique sabotée »	34 à 38	Binômes ou seul, jeu
15h50	5 min	Thanos en cinq minutes	39	Démonstration
15h55	5 min	Lundi matin	40	Tous ensemble
16h00		Fin, ferme		

Ce qui est sacrifié, et je le dis à 9h00 comme à 15h55 : écrire l'instrumentation, écrire du PromQL à la main, les recording rules (déjà en place), les droits Grafana, la sauvegarde, le TP Thanos complet. Tout est dans le dépôt, avec les guides des jours 1 à 3.

Les points de contrôle. En retard, je coupe dans cet ordre, sans remords :

1. L'étape 5 du TP A (crash test) : en démonstration.
2. Le TP B passe à 4 panels (CPU et mémoire, utilisation et saturation), plus l'import 1860.
3. La partie 5 du TP C (silence) : démonstration de 2 minutes.
4. Le *mute timing* du TP D : je le montre.
5. Thanos : une phrase et la slide, sans démonstration.

Je ne coupe **jamais** l'escape game ni le moment où les messages arrivent dans Teams : ce sont les deux souvenirs de la journée.

Les supports.

- Le deck **Jour3-Prometheus-Grafana.pptx**, 40 slides, avec le texte à dire dans les notes.
- Le **guide stagiaire jour 3** (PDF, et Markdown pour Notion), avec à la fin une synthèse de toute la formation, quatre checklists de mise en production, un déroulé type et les ressources.
- La branche `jour3` du dépôt : tout est préconfiguré, et la télécommande `./jour3.sh` fait le reste (`start` , `teams` , `mystere` , `solution` , `repare` , `thanos`).

La veille au soir (ou à 7h30) — 30 minutes

Pas à pas — pousser les branches sur GitHub

Le fichier `POUSSER-JOUR3.command` est dans le dossier `.../PROMETHEUS GRAFANA/2026/` sur mon Mac. Double-clic : il récupère le bundle et pousse les quatre branches (`jour3` , `jour3-formateur` , `formation-2026` , `formation-2026-formateur`). Si macOS refuse de l'ouvrir : clic droit → **Ouvrir** → **Ouvrir**. Ou bien, dans Terminal :

```
cd "/Users/yparent/Documents/PERSO/YSYCloud/FORMATION/PROMETHEUS GRAFANA/2026/formation-observabilite-lab"
git fetch ../formation-2026-jour3.bundle jour3:jour3 jour3-formateur:jour3-formateur formation-2026:formation-2026 formation-2026-formateur:formation-2026-formateur --update-head-ok --force
git push --force origin jour3 jour3-formateur formation-2026 formation-2026-formateur
```

Vérification sur github.com : le sélecteur de branches affiche `jour3`, qui contient `jour3.sh` à la racine et le guide `docs/stagiaire/Guide-stagiaire-Jour-3-pratique.pdf`.

Pas à pas — créer le workflow Teams de la salle 10 min

C'est ce qui permettra aux stagiaires d'envoyer de vrais messages dans Teams. Je le fais sur **mon** Teams (Sparks ou YSY Cloud), pas sur celui d'Apicil : je ne dépends pas de leur politique de sécurité, et je projette le canal.

1. Dans Teams, je crée une équipe ou j'utilise une équipe existante, et j'y crée un canal **standard** (pas privé) : `Formation Apicil - alertes`.
2. Sur le canal : `...` (Plus d'options) → **Workflows**.
3. Je cherche le modèle « **Send webhook alerts to a channel** » (le même modèle s'appelle « Post to a channel when a webhook request is received » dans Power Automate). Selon la langue de Teams, le libellé peut être traduit : je cherche « webhook ».
4. Nom du workflow : `Alertes formation`. **Suivant**. Je vérifie l'équipe et le canal. **Ajouter un workflow**.
5. Teams affiche une **adresse** (elle commence par `https://` et contient `workflows` ou `logic.azure.com`). Je la **copie** dans un fichier texte sur mon bureau : `url-teams.txt`.
6. Je teste depuis mon Mac, dans Terminal (en remplaçant l'adresse) :

```
curl -H "Content-Type: application/json" -d '{"type":"message","attachments":
[{"contentType":"application/vnd.microsoft.card.adaptive","content":
{"type":"AdaptiveCard","version":"1.4","body":[{"type":"TextBlock","text":"Test de la formation :
ça marche !"}]}]}' "COLLER-ICI-L-ADRESSE"
```

Le message « Test de la formation : ça marche ! » arrive dans le canal en quelques secondes.

Sécurité. Cette adresse permet à n'importe qui d'écrire dans le canal. Je la donne aux stagiaires pendant le TP C, et **je supprime le workflow ce soir** (Teams → le canal → `...` → Workflows → le workflow → Supprimer). Je le dis aux stagiaires : c'est une bonne pratique en soi.

Plan B si les workflows sont bloqués sur mon tenant : les stagiaires gardent l'adresse de l'Inbox (`http://inbox:8080/teams/grafana`), qui simule Teams. On perd l'effet « waouh », pas la compétence.

Pas à pas — préparer mon Codespace de démonstration 10 min

1. github.com → le dépôt → **Code** → **Codespaces** → `...` à côté de **Create codespace on...** → **New with options** : branche `jour3-formateur`, machine **4-core** si possible. **Create codespace**.
2. Dans le terminal : `./jour3.sh start`. J'attends `10 / 10 cibles UP`.
3. Les deux dashboards corrigés des jours précédents, comme exemples à montrer :

```
cp solutions/jour-2/dashboards/*.json grafana/dashboards/
```

4. Je copie l'alerting du TP C pour être prêt à le montrer :

```
cp jour3/alerting/alerts.yml prometheus/rules/alerts.yml
cp jour3/alerting/alertmanager.yml alertmanager/alertmanager.yml
./lab.sh check && ./lab.sh reload
```

5. Je teste le vrai Teams depuis le Codespace : `./jour3.sh teams 'ADRESSE'` , puis `./lab.sh chaos errors on` . Deux minutes plus tard, une carte « ShopHighErrorRate » arrive dans le canal. `./lab.sh chaos errors off` , puis `./jour3.sh teams off` .

Pas à pas — tester comme un stagiaire 5 min

Dans une fenêtre de navigation privée, avec mon compte GitHub : j'ouvre <https://codespaces.new/yparent/formation-observabilite-lab/tree/jour3>, je vérifie que la branche affichée est `jour3` , **Create codespace**, puis `./jour3.sh start` . Si j'obtiens `10 / 10 cibles UP` , la journée est sécurisée. Je supprime ce Codespace ensuite.

Pas à pas — le message à poster à 9h00

Je le prépare dans le chat de la session :

```
Bonjour à tous ! Pour démarrer :
1) Ouvrez ce lien (connecté à GitHub) et cliquez sur Create codespace :
  https://codespaces.new/yparent/formation-observabilite-lab/tree/jour3
2) Dans le terminal du Codespace : ./jour3.sh start
3) Attendez "10 / 10 cibles UP".
Le guide du jour : docs/stagiaire/Guide-stagiaire-Jour-3-pratique.pdf (dans le Codespace,
ou sur GitHub, branche jour3). Gardez un onglet ouvert sur l'IA de votre choix.
```

8h30 — Vérification

- Codespace de démo : `./jour3.sh status` . Grafana ouvert.
- Teams ouvert sur le canal `Formation Apicil - alertes` , prêt à être projeté.
- Le deck en mode Présentateur, slide 1.
- Onglets, dans l'ordre : le deck, Grafana, Prometheus, Alertmanager, Inbox, Mailpit (tous de mon Codespace de démo), une IA, Teams, le dépôt GitHub.
- Au tableau : le lien `codespaces.new/.../tree/jour3` et `./jour3.sh start` , en gros. À côté, un tableau vide à deux colonnes « Alertmanager | Grafana », pour 15h25.

9h00 — Démarrer (15 min) · slides 1 à 5

Slide 1 — accueil

« Bonjour à tous. Dernier jour. Hier, on s'est arrêté au moment d'écrire du code dans l'application. Aujourd'hui, on change de rythme : on ne code plus, on pratique. Ce soir, vous aurez construit deux tableaux de bord selon les méthodes de Google, envoyé des alertes dans Teams, dans Slack et par e-mail, et mené une enquête sur une boutique sabotée. »

Slide 2 — « Aujourd'hui, vous ne codez pas. Vous pilotez. »

« Vous êtes des gens d'infrastructure. Lundi, ce n'est pas vous qui instrumenterez les applications : ce sont vos développeurs. Et PromQL, une IA l'écrit très bien. Mais une IA se trompe avec beaucoup d'assurance. Donc la compétence d'aujourd'hui, ce n'est pas d'écrire du PromQL : c'est de le demander, de le **comprendre**, et de le vérifier. À chaque fois, vous demanderez à l'IA d'expliquer sa requête, ligne par ligne. »

Trois secondes de silence. La phrase donne le ton.

Slide 3 — le programme

Je lis la colonne de droite : « ce que vous aurez fait ». « Ce soir, on vérifiera ensemble. » Pausés 10h30 et 14h30, déjeuner 12h30, fin 16h00.

Slide 4 — démarrer, en même temps qu'eux

J'envoie le message préparé dans le chat.

Pas à pas — le démarrage, projeté

1. Dans mon navigateur : le lien `codespaces.new/yparent/formation-observabilite-lab/tree/jour3`. Je montre la page : le dépôt, la **branche jour3**, la machine 2 cœurs. **Create codespace**.
2. Pendant les deux minutes de création : « Le Codespace, c'est un ordinateur dans le cloud de GitHub, avec tout notre lab. Rien n'est installé sur votre poste. »
3. Le terminal apparaît en bas. Je tape `./jour3.sh start`. Je commente : - « Il démarre treize conteneurs : Prometheus, Grafana, l'application en deux exemplaires, le générateur de trafic, les exporters, l'Alertmanager, l'Inbox qui joue Teams et Slack, et Mailpit, un faux serveur de messagerie. »
- « À la fin, il vérifie lui-même que Prometheus voit ses dix cibles. » 4. `10 / 10 cibles UP` : je fais lever la main de ceux qui l'ont. 5. Onglet **PORTS** (à côté de *Terminal*) : je montre l'icône **globe** sur la ligne 3000. Grafana s'ouvre : `admin / formation`.

Ceux qui n'ont pas 10/10 :

Symptôme	Cause probable	Remède
Le Codespace s'ouvre sur <code>main</code> ou <code>formation-2026</code>	mauvais lien, ou branche changée	refaire avec le lien exact ; ou en bas à gauche de VS Code, cliquer sur la branche et choisir <code>jour3</code>
Docker ne répond pas	le Codespace finit de démarrer	attendre 30 s, relancer <code>./jour3.sh start</code>
<code>toomanyrequests</code>	limite de Docker Hub (Redis, Grafana)	<code>docker login</code> avec un compte Docker Hub gratuit, relancer
8 ou 9 cibles sur 10	un conteneur encore en démarrage	relancer <code>./jour3.sh start</code> : il est rejouable
<code>./jour3.sh: Permission denied</code>	droit d'exécution perdu	<code>bash jour3.sh start</code>
Le port 3000 ne s'ouvre pas	le navigateur bloque les fenêtres	onglet PORTS → clic droit → <i>Open in Browser</i>
Plus de quota Codespaces (60 h/mois gratuites)	compte très utilisé	un binôme avec le voisin

Slide 5 — les commandes pour casser

« Ces commandes vont servir toute la journée. Un tableau de bord ou une alerte qu'on n'a jamais vus réagir à une panne, on ne sait pas s'ils marchent. Les pompiers font des exercices d'incendie ; nous, on fait du chaos. Et quand vous êtes perdus : `./jour3.sh repare` . »

9h15 — Les bonnes métriques et PromQL avec l'IA (25 min) · slides 6 à 13

Slide 6 — section

« Première question : qu'est-ce qu'on met dans un tableau de bord ? La mauvaise réponse : "toutes les métriques qu'on a". La bonne : on part d'une méthode. Et la plus connue vient de Google. »

Slides 7 et 8 — les quatre signaux dorés 5 min

« En 2016, Google publie gratuitement le livre qui décrit comment ses équipes d'exploitation travaillent : *Site Reliability Engineering*, sur sre.google. Le chapitre 6, *Monitoring Distributed Systems*, dit en substance : si vous ne pouvez mesurer que quatre choses sur un service utilisé par des gens, mesurez la latence, le trafic, les erreurs et la saturation. »

Source, à montrer si quelqu'un veut la lire : <https://sre.google/sre-book/monitoring-distributed-systems/>

Je détaille avec **la voiture** : la vitesse (le trafic), le voyant moteur (les erreurs), le temps de trajet (la latence), la jauge d'essence (la saturation). « Quatre informations, et on conduit. Le reste, c'est pour le garagiste. »

Les deux subtilités, à dire :

- « La latence des erreurs à part. Une erreur 500 renvoyée en 2 millisecondes fait baisser la latence moyenne : on croit que ça va mieux, alors que ça va plus mal. »
- « La saturation est le seul signal qui prévient. Les trois autres constatent. »

Le schéma du bas (slide 7) introduit déjà USE : « Quand c'est rouge en haut, on descend voir la machine. »

Slide 9 — signaux dorés, RED, USE 3 min

« RED, c'est la version microservices de Tom Wilkie : les trois premiers signaux, sans la saturation, si simple qu'on peut exiger le même dashboard pour les 200 services de l'entreprise. USE, c'est l'autre côté : pas le service, la ressource. Brendan Gregg, l'ingénieur performance de Sun puis de Netflix : pour chaque ressource, l'utilisation, la saturation, les erreurs. »

Au tableau, deux étages : en haut « le service » (signaux dorés), en bas « la machine » (USE), et une flèche de haut en bas. « Ce matin, TP A, l'étage du haut. TP B, l'étage du bas. Un lien entre les deux. On alerte sur le haut, on diagnostique avec le bas. »

Sources : <https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/> et <https://www.brendangregg.com/usemethod.html>

Slides 10 à 12 — demander, comprendre, vérifier 4 min

« Trois étapes. **Demander**, avec le contexte et les vraies métriques, sinon l'IA invente des noms plausibles qui n'existent pas. **Comprendre** : l'IA doit expliquer sa requête, ligne par ligne, et vous devez pouvoir la redire en une phrase à votre voisin. **Vérifier** : quatre contrôles, dont le plus important, je casse et je regarde la courbe bouger. »

Sur la slide 11, je souligne les deux dernières lignes du modèle : « Je demande la requête, **puis l'explication**. Ce n'est pas de la politesse, c'est l'objectif. Une requête qu'on ne comprend pas, on ne saura pas la réparer le jour où elle sera fausse. »

La règle de sécurité, lentement : « Chez un assureur, on ne colle jamais de données de production dans une IA publique : noms de clients, adresses internes, mots de passe. Ici, la boutique est fictive. Lundi, demandez à votre RSSI quel outil d'IA est autorisé. »

Pas à pas — la démonstration en direct 5 min

1. J'ouvre `...5001/metrics` dans mon Codespace de démo, `Ctrl+F` `# TYPE http_requests_total`, je copie les deux lignes `# HELP` et `# TYPE` et trois lignes de valeurs.
2. Dans l'onglet IA, je colle le modèle de leur guide et je remplace la dernière ligne entre chevrons par « le taux d'erreur 5xx en pourcentage, pour toute la boutique ».

3. Je lis l'explication à voix haute, en m'arrêtant sur chaque fonction : `rate` (la vitesse du compteur), `sum` (on additionne les instances et les routes), `status=~"5.."` (la regex des codes 500 à 599), la division (la part des erreurs), `* 100` (en pourcentage).
4. Je reformule en une phrase : « Elle calcule, pour toute la boutique, la part des requêtes en erreur sur les dernières minutes. » « Voilà le test de compréhension. »
5. Grafana → **Explore** → Prometheus → mode **Code**. Je colle la requête (`$__rate_interval` devient `5m` : c'est une variable de dashboard).
6. Les quatre vérifications, à voix haute : ça s'exécute ; l'ordre de grandeur (0, la boutique est saine) ; le compteur est dans un `rate` ; puis `./lab.sh chaos errors on`, j'attends une minute, la courbe monte. « Maintenant, elle est vérifiée. » `./lab.sh chaos errors off`.

Si l'IA se trompe pendant la démonstration, **tant mieux** : je lui redonne le résultat, elle se corrige, et c'est la meilleure démonstration du « faux ? on redonne l'erreur ». Les erreurs classiques : un nom inventé (`http_server_requests_seconds_count`, le nom Spring Boot), la moyenne à la place du p95, un `sum` avant le `rate`, `status="500"` au lieu de `~"5.."`.

Slide 13 — l'échauffement 7 min

« À vous : une requête par signal doré, pour toute la boutique, dans Explore. L'IA l'écrit et l'explique, vous la vérifiez, et vous écrivez avec **vos** mots ce qu'elle fait. Sept minutes. »

Je circule. À la fin, je fais lire deux explications à voix haute.

Signal	Ordre de grandeur	Erreurs d'IA fréquentes
Trafic	6 à 7 req/s	le compteur brut sans <code>rate</code> (des milliers, qui montent)
Erreurs	0 sans chaos	<code>status="500"</code> ; pas de division (un débit, pas un taux)
Latence p95	0,15 à 0,25 s	la moyenne <code>_sum / _count</code> ; <code>sum by (route)</code> sans <code>le</code> : « No data »
Saturation	quelques % de CPU	sans <code>rate</code> ; oubli du <code>1 -</code> (donne le CPU libre)

9h40 — TP A : la boutique en quatre signaux dorés (50 min + 40 min) · slides 14 à 18

Slides 14 à 16 3 min

Slide 15 (choisir la visualisation) : « Votre antisèche pour la journée, je la laisse au tableau. » Slide 16 (lisible) : « Un dashboard sans unité, c'est une réunion où quelqu'un dit "on est à 0,4" et où personne n'ose demander 0,4 quoi. »

Slide 17 — le TP

« Votre dashboard a trois étages, comme un immeuble. Au rez-de-chaussée, en haut de l'écran, la réponse à "ça va ?" : quatre cases de couleur, une par signal doré. Au premier, les mêmes signaux dans le temps, pour l'équipe technique. Au deuxième, le métier, pour le directeur commercial. Chaque panel a sa question en français dans votre guide : vous la posez à l'IA, elle explique, vous vérifiez, vous réglez l'affichage. Bloqués plus de cinq minutes sur une requête : l'annexe de secours. Personne ne reste bloqué sur du PromQL aujourd'hui : le sujet, c'est Grafana. »

Slide 18 : l'exemple de résultat, que je laisse projeté. Mieux : j'ouvre « TP 5 - Boutique en ligne » dans mon Grafana de démo.

Pas à pas — la variable (je la montre, 3 min)

1. **Dashboards** → **New** → **New dashboard**, mode édition : **Add** → **Variable** (ou **Settings** → **Variables** → **New variable**).
2. *Variable type* : Query. *Name* : `instance` . *Label* : `Instance` .
3. *Query* : `label_values(http_requests_total{job="shop-api"}, instance)` .
4. **Multi-value** et **Include All option** ; *Custom all value* : `.*` .
5. *Preview of values* : `shop-api-1:5000` , `shop-api-2:5000` . **Back to dashboard**.

« Dans chaque requête, `instance=~"$instance"` . Le `=~` parce que "All" devient `.*` , une regex. C'est dans le modèle de demande : l'IA le fera. »

Pas à pas — le premier Stat, de bout en bout (je le montre, 4 min)

1. **Add** → **Visualization**, source *Prometheus*, mode **Code**, la requête du trafic (vérifiée).
2. En haut à droite, visualisation **Stat**.
3. Options, champ de recherche : `unit` → *requests/sec (rps)*.
4. `color mode` → **Background** ; `graph mode` → **Area**.
5. **Thresholds** → *Color scheme* : couleur unique bleue. « Le trafic n'est ni bon ni mauvais : il ne doit jamais être rouge. Le rouge, c'est "il faut agir". »
6. Titre `Trafic` , **Back to dashboard**, je le réduis à un quart de largeur. **Ctrl+S**.

Pendant le TP : les pièges, dans l'ordre où ils arrivent

Piège	Symptôme	Ce que je dis
Unité oubliée	0.012 au lieu de 1,2 %	« Percent (0.0-1.0) pour un ratio, Percent (0-100) si la requête multiplie par 100. »
Seuils à l'envers	tout est rouge	ils se lisent de bas en haut : <i>Base</i> vert, orange à 0.01, rouge à 0.05

Piège	Symptôme	Ce que je dis
Latence « No data »	<code>sum by (route)</code> sans <code>le</code>	« Le <code>le</code> , c'est l'escalier des buckets : sans lui, plus d'escalier. »
Heatmap grise	<i>Format : Heatmap</i> oublié	sous la requête : <i>Options → Format → Heatmap</i> , puis <i>Calculate from data : No</i>
Donut d'une seule part	pas de <code>by (payment_method)</code>	et la légende <code>{{payment_method}}</code>
Stock : 12 barres	pas d' <code>avg by (product)</code>	les deux instances ont chacune leur stock
Rien n'est sauvegardé	tout perdu au rechargement	« Ctrl+S toutes les dix minutes. »

La question de l'étape 3 : pourquoi `rate` pour un montant ? `shop_revenue_euros_total` est un compteur : tout ce qui a été vendu depuis le démarrage, remis à zéro à chaque redémarrage. `rate` × 3600 donne la **vitesse** en euros par heure. Le compteur kilométrique contre le compteur de vitesse.

À 10h25 : « Là où vous en êtes, sauvegardez. Après la pause : le métier et les finitions. »

10h30 — Pause 30 min

11h00 — TP A, suite : les finitions (je les montre, 5 min)

- Le panel Text** : *Add → Visualization → Text*, mode Markdown, trois lignes (voir le bloc ci-dessous).
Je dis : « Le premier lecteur de ce dashboard, c'est quelqu'un d'astreinte, réveillé à 3 h, qui ne l'a jamais vu. Trois lignes lui disent où il est et qui appeler. »
- L'annotation Chaos** : *Add → Annotation query*. Je lance `./lab.sh chaos errors on` : un trait rouge vertical sur toutes les courbes. « En post-mortem, la première question est toujours : qu'est-ce qui a changé à ce moment-là ? »
- Le lien** vers le dashboard serveur, *Keep time range*. « Quand le haut est rouge, un clic pour descendre d'un étage, sur la même période. »

Le texte du panel Text :

```
**Boutique en ligne : santé et ventes.**
Propriétaire : équipe Boutique · Astreinte : canal Teams #boutique-astreinte
En cas d'alerte : [runbook](https://github.com/yparent/formation-observabilite-lab)
```

Anecdote — le dashboard du directeur. On m'avait demandé « un dashboard pour le directeur ». J'en ai livré un de 40 panels, magnifique. Il a regardé, et il a demandé : « Donc, ça va ou pas ? » Depuis, la première rangée de chaque dashboard répond à cette question, en quatre chiffres colorés.

Le crash test (étape 5) : `./lab.sh traffic 30` . La réponse attendue à « est-ce une panne ? » : **non**. Le trafic, la saturation et le chiffre d'affaires montent, les erreurs et la latence restent vertes. « C'est le Black Friday qui se passe bien. Si le trafic avait une case rouge, on aurait réveillé quelqu'un pour une bonne nouvelle. »

Le joker, pour celui qui est perdu (sur la branche `jour3` , les corrigés sont dans la branche `formateur`) :

```
git fetch origin jour3-formateur
git checkout origin/jour3-formateur -- solutions/jour-2/dashboards/
cp solutions/jour-2/dashboards/*.json grafana/dashboards/
```

Dix secondes plus tard, deux dashboards complets sont dans son dossier *Formation*.

11h40 — TP B : le serveur en méthode USE (30 min) · slide 19

« Même exercice pour la machine. Pour chaque ressource, trois questions : est-elle occupée ? Est-ce que du travail attend ? Y a-t-il des erreurs ? La méthode de Brendan Gregg vous empêche d'oublier une ressource. »

La saturation CPU (panel 2) et le piège de l'IA. La charge compte les processus qui veulent le CPU : 2 sur une machine à 2 cœurs, c'est plein ; 4, la moitié attend. D'où « charge divisée par le nombre de cœurs ». L'IA propose presque toujours `node_load1 / count(node_cpu_seconds_total{mode="idle"})` , qui renvoie « No data » : à gauche une série avec les labels `instance` et `job` , à droite un `count` sans aucun label ; PromQL apparie par labels identiques, rien ne correspond. La correction : `/ on (instance) count by (instance) (...)` .

« C'est le meilleur exercice de la journée : donnez le résultat à l'IA, demandez-lui d'expliquer. Travailler avec une IA, c'est une conversation, pas un distributeur. »

La pression (PSI, panel 3). Le noyau Linux mesure directement le temps pendant lequel des tâches **attendent** le CPU : la saturation à l'état pur. « Si vous ne reprenez qu'une métrique de saturation pour Linux, c'est celle-là. »

MemAvailable et pas MemFree : Linux utilise la mémoire libre comme cache disque et la rend à la demande. `MemFree` est toujours bas et fait peur pour rien.

Anecdote — le serveur à 98 % de mémoire. Des années de tickets « mémoire critique » sur un serveur de base de données, parce que la supervision regardait `MemFree` . Le jour où on est passé à `MemAvailable` , les alertes ont disparu, et personne ne les a regrettées.

La chaîne complète, vers 12h00 : `./lab.sh chaos cpu 120` . La case Saturation de la boutique passe à l'orange, un clic sur le lien, le dashboard serveur montre le CPU, la charge, la pression. « Un diagnostic en deux clics. »

L'import du 1860 : Dashboards → New → Import, `1860`, *Load*, source Prometheus, *Import*. « Le dashboard le plus téléchargé de grafana.com. Parfait pour une enquête de deux heures. À 3 h du matin, vous ouvrez le vôtre. »

12h10 — Auto-audit et maturité (20 min) · slides 20 et 21

Slide 20, le modèle de maturité de Grafana (2 min) : « Faible, moyen, élevé. Chez vous ? » Réponse habituelle : faible, avec un sourire. « Lundi, vous pouvez passer à moyen en une semaine. »

Slide 21 :

« Dix critères, tirés de la documentation de Grafana et du chapitre 6 de Google SRE. Notez votre dashboard Boutique, honnêtement. Puis échangez d'ordinateur avec votre voisin et notez le sien. Huit minutes. »

Débrief (5 min) : « Qui a 8 ou plus ? » Les deux critères les plus ratés : 8 (annotations) et 9 (lien vers le niveau suivant). « Ce sont les deux qui font gagner le plus de temps pendant un incident. » Le reste du temps : ils corrigent leur dashboard.

Source : <https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/best-practices/>

12h30 — Déjeuner

« Laissez vos Codespaces ouverts. S'ils se mettent en veille, ils redémarrent avec la stack au retour : 30 secondes. »

Pour moi, à 13h25 : dans mon Codespace de démo, `./jour3.sh thanos on`, pour que Thanos ait des données à 15h50.

13h30 — Alerter sans épuiser les équipes (10 min) · slides 22 à 27

- **Slide 23 — Mauvaise alerte, bonne alerte.** « Google SRE, et la documentation de Prometheus le dit en une phrase : *keep alerting simple, alert on symptoms*. On alerte sur ce que les utilisateurs sentent. Et une alerte qui réveille quelqu'un doit être urgente et actionnable : si la réponse à "qu'est-ce que je fais ?" est "je regarde", c'est un panel. » Source : <https://prometheus.io/docs/practices/alerting/>
- **Slide 24 — Le cycle de vie.** Inactive, Pending, Firing. « Le `for`, c'est l'anti-faux positif : le détecteur de fumée qui sonnerait au premier grille-pain, tout le monde finirait par enlever la pile. »
- **Slide 25 — Deux chemins.** « Même canaux, deux chemins. Jamais la même alerte des deux côtés. »
- **Slide 26 — Les concepts.** « Retenez ce tableau : vous allez le vivre dans les deux TP. »
- **Slide 27 — L'arbre de routage.** « Un aiguillage de gare. Le `continue`, c'est la photocopie. »

Anecdote — les 400 notifications par jour. Une équipe recevait 400 notifications par jour. Plus personne ne les lisait ; la nuit où la base est tombée, l'alerte était la 237e de la journée. Après nettoyage : 12 alertes, toutes sur des symptômes, chacune avec un runbook.

13h40 — TP C : Prometheus, Alertmanager, Teams, Slack, e-mail (50 min) · slides 28 à 30

Les fichiers sont **fournis** dans `jour3/alerting/`. Personne n'écrit 80 lignes de YAML.

Partie 1 — Les règles 10 min

Je projette `jour3/alerting/alerts.yml` et je lis `ShopHighErrorRate` à voix haute :

« Une règle, c'est quatre choses. Une requête qui renvoie quelque chose quand ça va mal : le tamis, tout ce qui dépasse 5 % passe. Un `for` : ça doit durer une minute. Des labels : `severity` et `team`, c'est l'adresse sur l'enveloppe, l'Alertmanager ne lit que ça. Des annotations : le texte de la lettre, avec le lien vers le runbook. Si une expression vous échappe, collez-la dans l'IA et demandez l'explication. »

Correction des questions :

- Symptômes : `ShopHighErrorRate`, `ShopCheckoutSlow`, `BlackboxProbeFailed`, et `TargetDown` (l'absence de mesure). Cause : `HostHighCpuLoad`, et c'est pour ça qu'elle est en `warning`.
- `for` : l'anti-faux positif, le passage *Pending* → *Firing*. `team` : sert au routage.
- Réveille : `ShopHighErrorRate` (critical). Ne doit jamais réveiller : `HostHighCpuLoad`.

Partie 2 — L'arbre de routage 10 min

Je dessine l'arbre **au tableau** à partir de `amtool config routes` :

```
racine → inbox-default
├── severity=critical → astreinte-teams (continue: true)
├── team=boutique → boutique (Slack + e-mail)
└── team=infra → infra-inbox
```

Correction des prédictions :

Labels	Receivers
<code>severity=critical team=boutique</code>	<code>astreinte-teams</code> , <code>boutique</code>
<code>severity=warning team=boutique</code>	<code>boutique</code>
<code>severity=warning team=infra</code>	<code>infra-inbox</code>
<code>severity=critical (sans team)</code>	<code>astreinte-teams</code>
<code>severity=info team=logistique</code>	<code>inbox-default</code> (aucune branche : la racine)

Sans `continue: true`, la première ligne ne donne plus que `astreinte-teams` : l'équipe boutique ne sait pas que son site est en panne. « L'erreur de routage la plus fréquente en production. On teste son routage **avant** la panne : `amtool config routes test`. »

Un receiver, deux canaux. Je montre le receiver `boutique` : un bloc `slack_configs` et un bloc `email_configs`. « Une équipe, deux canaux. »

Partie 3 — Casser et suivre les messages 10 min

`./lab.sh chaos errors on`. Chronologie attendue : *Pending* en 15 à 30 s ; *Firing* une minute après ; notifications 10 s plus tard (`group_wait`).

- **Inbox** (8080) : une carte **Teams** (astreinte) et un message **Slack** (boutique), à cause du `continue`. Les deux instances dans le **même** message : `group_by: [alertname, job]`.
- **Mailpit** (8025) : un e-mail `[FIRING:2] ShopHighErrorRate` à `equipe-boutique@boutique.local`. Je l'ouvre projeté : le résumé, les labels, le lien vers l'Alertmanager. « Exactement ce que recevrait l'équipe. »
- Le *resolved* arrive jusqu'à une minute après la fin du chaos (`group_interval: 1m` ; souvent 5 minutes en production).

Partie 4 — L'inhibition 10 min

« Le site est en erreur, l'astreinte est réveillée. A-t-elle besoin, en plus, de "le paiement est lent" ? Non : c'est du bruit. L'inhibition, c'est "si A sonne, fais taire B". »

Ils ajoutent le bloc de six lignes, `check`, `reload`, `chaos errors` **et** `latency`. Au bout de deux minutes, `amtool alert query --inhibited` montre `ShopCheckoutSlow` (*suppressed*), et rien n'est arrivé pour elle dans l'*Inbox* ni dans Mailpit.

Partie 5 — Le silence 5 min

Démonstration dans l'Alertmanager (9093) : **New Silence**, matcher `team="boutique"`, 30 minutes, commentaire « Maintenance base de données ». « L'inhibition est une règle permanente ; le silence est ponctuel, posé par un humain, avec son nom et une raison. »

Partie 6 — Le vrai Teams (démonstration, 5 min) · slide 29

« Point important pour vous, qui utilisez Teams. Pendant des années, on créait un "connecteur Incoming Webhook". Microsoft l'a remplacé par les **workflows**. Dans Teams : le canal, les trois points, Workflows, le modèle "Send webhook alerts to a channel". Teams donne une adresse, et cette adresse est un **secret** : qui l'a peut écrire dans le canal. »

Pas à pas — le vrai Teams, projeté

1. Dans mon Codespace de démo : `./jour3.sh teams 'ADRESSE-DU-WORKFLOW'` (avec les apostrophes : l'adresse contient des `&`).

2. Je montre le fichier : `cat alertmanager/secrets/teams_url` . « L'adresse est dans un fichier à part, que Git ignore. Le YAML dit seulement `webhook_url_file` . »
3. `./lab.sh chaos errors on` . Je bascule sur Teams, projeté. Une à deux minutes plus tard : la carte « ShopHighErrorRate » arrive dans le canal. Effet garanti.
4. `./lab.sh chaos errors off` , puis `./jour3.sh teams off` .

Je donne l'adresse dans le chat de la session : « Ceux qui veulent peuvent faire pareil depuis leur Codespace. Et cet après-midi, vous l'utiliserez tous depuis Grafana. »

Slide 30 — l'e-mail (2 min) : « Au lab, Mailpit. En production, le relais SMTP de l'entreprise, avec TLS et un compte de service. L'e-mail pour le non urgent ; le critique dans Teams ou l'outil d'astreinte : personne ne lit ses e-mails à 3 h du matin. Des listes de diffusion par équipe, jamais des adresses nominatives. »

À 14h28 : « `./jour3.sh repare` , et pause. »

14h30 — Pause 30 min

15h00 — TP D : l'alerting de Grafana, Teams en vrai (30 min) · slides 31 à 33

« Le directeur commercial ne lira jamais un fichier YAML, et il veut changer son seuil lui-même. On lui fait son alerte dans Grafana, par e-mail. Et l'astreinte la verra dans Teams. On ne refait **pas** les alertes techniques du TP C : la même alerte des deux côtés, c'est deux notifications, deux vérités, et un jour l'une des deux est fausse. »

Pas à pas — le premier contact point, projeté 2 min

Alerting → Notification configuration → Contact points → + Create contact point, nom `equipe-commerce` , intégration **Email**, adresse `commerce@boutique.local` , **Test** : l'e-mail arrive dans Mailpit. **Save**.

Le moment Teams 3 min

« Maintenant, chacun crée un contact point **Microsoft Teams**, nommé `teams-salle` , avec l'adresse que je viens de mettre dans le chat. Et vous cliquez sur **Test**. Regardez l'écran. »

Je projette Teams. Les messages de test arrivent les uns après les autres. Je laisse vivre.

Pendant le TP : les pièges

Piège	Ce que je dis
Le test Teams échoue	l'adresse est incomplète (copier-coller tronqué) : la recopier depuis le chat
La règle reste en <i>Normal</i>	le seuil : <i>IS BELOW</i> la moitié de la valeur actuelle (environ 170 000 €/h au lab), puis <code>./lab.sh traffic 1</code>
<i>Preview</i> montre des courbes	requête en <i>Range</i> : passer en Instant
L'e-mail ne part pas, l'alerte va dans Teams	le label <code>team=commerce</code> manque sur la règle, ou le matcher de la politique est mal saisi
Rien après <i>Firing</i>	<i>Group wait</i> de 30 s par défaut, plus l'intervalle d'évaluation : 1 à 2 minutes

Le déclenchement : `./lab.sh traffic 1`. Le chiffre d'affaires tombe d'environ 170 000 à 20 000 €/h en deux minutes, la règle passe *Pending* puis *Firing*, l'e-mail arrive dans Mailpit. `./lab.sh traffic 6`.

Le tableau comparatif, tous ensemble 7 min

Je remplis au tableau, **avec leurs réponses** :

	Prometheus + Alertmanager	Grafana
Où vit la configuration ?	des fichiers YAML	la base de Grafana (ou des fichiers de provisioning)
Versionner, relire	Git, merge request, naturellement	export, provisioning, Git Sync : possible, moins naturel
Tester avant de déployer	<code>promtool test rules</code> , <code>amtool config routes test</code>	<i>Preview</i> , <i>Test</i> sur le contact point
Qui modifie ?	ceux qui touchent au YAML	quiconque a les droits sur le dossier
Plusieurs sources (SQL, logs)	non, métriques Prometheus seulement	oui
Si Grafana tombe	les alertes continuent	plus d'alertes
Inhibition	oui	pas d'équivalent direct

Slide 32, la conclusion : « Les alertes techniques, critiques, qui doivent survivre à tout : Prometheus et Alertmanager, dans Git, testées. Les alertes métier, multi-sources : Grafana. »

Slide 33, le budget d'erreur, en une minute (texte dans les notes) : « Le niveau suivant. » Source : <https://sre.google/workbook/alerting-on-slos/>

15h30 — Bonnes pratiques et escape game (20 min) · slides 34 à 38

Slides 35 et 36 — les bonnes pratiques 3 min

Je les passe vite : « Six règles de configuration, cinq règles de sécurité par outil. Vous les retrouvez en **checklists à cocher** à la fin de votre guide, avec un déroulé type de mise en production. C'est ce que vous emporterez. »

Sources : <https://prometheus.io/docs/operating/security/> et <https://grafana.com/docs/grafana/latest/setup-grafana/configure-security/>

Slides 37 et 38 — l'escape game 17 min

« Cette nuit, quelqu'un a saboté la boutique. Deux incidents se cachent. Vous êtes l'astreinte. Tapez `./jour3.sh mystere`, puis fermez le terminal. Interdit : le terminal, docker, les logs. Autorisé : vos dashboards, Explore, les alertes, l'Inbox, Mailpit, et l'IA. Douze minutes. Remplissez le rapport d'enquête. »

Chaque Codespace tire **deux** sabotages au hasard parmi **sept** : les voisins n'ont pas les mêmes. Je chronomètre au tableau. À 12 minutes : « Stop. `./jour3.sh solution`. » Chacun voit sa solution, la boutique est réparée, ils comptent leurs points.

Les solutions des sept sabotages

N°	Sabotage	Où ça se voit	Alerte et canal	Le piège
1	Latence ×10 sur <code>shop-api-1</code> seulement	case Latence orange, heatmap ; variable <code>instance</code> sur <code>shop-api-1</code>	<code>ShopCheckoutSlow</code> (warning) → Slack + e-mail	la moyenne des deux instances dilue : filtrer par instance
2	40 % d'erreurs sur <code>shop-api-2</code> seulement	case Erreurs, environ 15-20 % au global, 30-40 % sur <code>shop-api-2</code>	<code>ShopHighErrorRate</code> sur <code>shop-api-2</code> → Teams + Slack + e-mail	une seule instance : le <code>by (instance)</code> de la règle la désigne
3	Redis arrêté	Explore : <code>redis_up = 0</code>	aucune	<code>up{job="redis"}</code> reste à 1 : c'est l'exporter qui répond. L'alerte qui manque : <code>redis_up == 0</code>
4	Trafic ×5	cases Trafic et Saturation, chiffre d'affaires en hausse	éventuellement <code>HostHighCpuLoad</code> → infra	ce n'est pas une panne : la bonne réponse est « beaucoup de clients »

N°	Sabotage	Où ça se voit	Alerte et canal	Le piège
5	CPU saturé pendant 15 min	case Saturation rouge, dashboard USE (CPU, charge, pression)	<code>HostHighCpuLoad</code> (warning) → infra-inbox	la boutique reste rapide : une saturation n'est pas encore une panne
6	<code>shop-api-1</code> arrêtée	State timeline, <code>up = 0</code> ; <code>probe_success = 0</code>	<code>TargetDown</code> et <code>BlackboxProbeFailed</code> → Teams	le trafic total baisse peu : l'autre instance absorbe
7	Fuite mémoire	Explore : <code>process_resident_memory_bytes{job="shop-api"}</code> qui grimpe	aucune	aucun dashboard ne le montre. L'alerte qui manque : une <code>predict_linear</code> sur la mémoire du processus

Le bonus de 2 points va à ceux qui trouvent le 3 ou le 7 et disent quelle alerte aurait dû exister. « C'est exactement le travail d'une revue post-incident : qu'est-ce qui nous a manqué ? »

Les questions éclair (réponses)

Je pose les questions à voix haute ; premier qui lève la main, 1 point.

1. **Le type de `http_requests_total`** : un compteur. On ne le regarde jamais brut : il ne fait que monter depuis le démarrage, et repart à zéro au redémarrage ; on lit sa vitesse, `rate`.
2. **Pourquoi le p95** : la moyenne ne décrit personne (95 clients à 50 ms et 5 à 10 s donnent 550 ms) ; le p95 est une promesse qu'on peut faire à 95 % des clients.
3. **Le `for`** : la condition doit tenir cette durée avant de notifier ; c'est l'anti-faux positif.
4. **`continue: true`** : l'alerte continue de descendre l'arbre après une branche qui correspond ; plusieurs receivers la reçoivent.
5. **Inhibition ou silence** : l'inhibition est une règle permanente de la configuration ; le silence est une décision humaine, ponctuelle, avec un auteur et une durée.
6. **L'adresse Teams dans un fichier** : c'est un secret (qui l'a peut écrire dans le canal), et le YAML part dans Git.
7. **`MemAvailable`** : Linux utilise la mémoire libre comme cache ; `MemFree` est toujours bas et fait peur pour rien.
8. **SQL** : Grafana (multi-sources). **Survivre à une panne de Grafana** : Prometheus et Alertmanager.

Le binôme gagnant : applaudissements.

15h50 — Thanos en cinq minutes · slide 39

Sur mon Codespace de démo, où Thanos tourne depuis 13h25.

« Un Prometheus garde quelques semaines, sur un seul serveur. Trois besoins arrivent un jour : un an d'historique ; une vue globale sur plusieurs Prometheus ; deux Prometheus jumeaux pour la haute disponibilité, sans tout voir en double. Thanos répond aux trois, sans toucher aux Prometheus : on leur colle un **sidecar**. »

Pas à pas — la démonstration

1. Port **10902**, **Stores** : deux sidecars (`replica=prom-1` , `replica=prom-2`) et le Store Gateway.
2. `up{job="shop-api"}` : **2 séries**. Je décoche **Use Deduplication** : **4**. « Le Querier sait que `replica` distingue les jumeaux, il garde une seule copie. »
3. `docker compose exec thanos-store ls /bucket` : un dossier par bloc de 10 minutes. « En production : un bucket S3 ou OVH Object Storage, quelques euros par mois pour des années. »
4. Grafana → Explore → source **Thanos** : la même requête. « Vos dashboards pointent sur Thanos sans rien changer d'autre. »

« Les alternatives : Mimir, VictoriaMetrics. Même API : Grafana ne voit pas la différence. À essayer chez vous : `./jour3.sh thanos on .` »

15h55 — Lundi matin · slide 40

« Lundi matin, ne faites pas tout. Choisissez **un** service. Un seul. Ses quatre signaux dorés en haut d'un dashboard. **Une** alerte sur un symptôme, avec un responsable, un runbook, et le bon canal Teams. Dans un mois, vous aurez dix services, parce que vos collègues viendront vous demander le même. »

« À la fin de votre guide : une synthèse de toute la formation, quatre checklists de mise en production, un déroulé type, et toutes les ressources : les sources de Google, les bonnes pratiques officielles, des petits outils comme PromLens qui explique une requête visuellement. Gardez-le. »

Tour de table éclair : un service chacun, à voix haute.

« Mardi matin, vous ne saviez pas ce qu'était une série temporelle. Aujourd'hui, vous avez construit deux tableaux de bord selon les méthodes de Google, envoyé des alertes dans Teams et par e-mail, et retrouvé deux pannes sans toucher au terminal. Merci à tous. »

16h00 : fin. **Ce soir** : je supprime le workflow Teams.

Annexe A — La télécommande `jour3.sh`

Commande	Ce qu'elle fait
<code>./jour3.sh start</code>	démarre les treize conteneurs, pousse le batch, vérifie les dix cibles, affiche les adresses
<code>./jour3.sh status</code>	état des conteneurs et adresses
<code>./jour3.sh teams 'URL'</code>	écrit l'adresse du workflow dans <code>alertmanager/secrets/teams_url</code> et recharge l'Alertmanager
<code>./jour3.sh teams off</code>	revient à l'Inbox du lab
<code>./jour3.sh mystere</code>	répare, puis applique deux sabotages au hasard parmi sept ; la solution est stockée encodée dans <code>.mystere</code>
<code>./jour3.sh solution</code>	affiche la solution, puis répare
<code>./jour3.sh repare</code>	redémarre <code>redis</code> , <code>shop-api-1</code> , <code>shop-api-2</code> (ce qui coupe aussi un chaos CPU), trafic à 6 req/s
<code>./jour3.sh thanos on / off</code>	active ou retire la brique Thanos, ses flags et la source de données Grafana

Annexe B — Ce que contient la branche `jour3`

Élément	État
<code>docker-compose.yml</code>	briques 01 à 06 et 09 (Mailpit) actives ; 07 (Thanos) et 08 (cAdvisor) commentées
<code>apps/shop-api/app.py</code>	les cinq TODO du TP 2 faits
<code>prometheus/prometheus.yml</code>	six jobs, recording rules, Alertmanager branché
<code>prometheus/rules/recording.yml</code> et <code>tests/</code>	les règles et les tests du TP 3
<code>alertmanager/alertmanager.yml</code>	la configuration de départ (une route vers l'Inbox) ; le TP C la remplace
<code>jour3/alerting/</code>	les règles et l'Alertmanager du TP C (Teams, Slack, e-mail)
Grafana	SMTP vers Mailpit (<code>GF_SMTP_*</code>), source Prometheus provisionnée
<code>docs/stagiaire/</code>	les guides, dont <code>Guide-stagiaire-Jour-3-pratique.pdf</code> et <code>jour-3-pratique.md</code>

La branche `jour3-formateur` contient en plus les corrigés (`solutions/`), le deck et ce guide.

Annexe C — Dépannage express

Symptôme	Remède
Une cible reste DOWN	<code>./jour3.sh start</code> à nouveau ; sinon <code>docker compose ps</code> et <code>docker compose logs <service></code>
Grafana ne montre aucune donnée	Explore → <code>up</code> : si vide, Prometheus ne tourne pas (<code>./jour3.sh start</code>)
Aucune notification dans l'Inbox	<code>./lab.sh check</code> (erreur de YAML ?), puis <code>./lab.sh reload</code> ; Prometheus → Status → Alertmanager discovery
L'e-mail n'arrive pas dans Mailpit	<code>docker compose ps mailpit</code> ; dans Grafana, <i>Test</i> sur le contact point
Le message n'arrive pas dans Teams	l'adresse est tronquée ; tester avec le <code>curl</code> de la veille ; plan B : l'Inbox
<code>./jour3.sh solution</code> dit « aucun sabotage »	le stagiaire a relancé <code>mystere</code> puis <code>solution</code> deux fois : <code>./jour3.sh repare</code>
Le Codespace est très lent	2 cœurs, c'est juste avec Thanos : <code>./jour3.sh thanos off</code>